

# Explain Product Binary Search

Yes — **the same high-level logic applies**, but there is one important difference.

Your previous problem was:

$$A[i] + B[j]$$

Now it is:

$$A[i] \times B[j]$$

The binary-search idea remains exactly the same:

**Guess an answer**  $x$ , count how many products are  $\leq x$ , and binary search the smallest  $x$  for which the count is at least  $k$ .

The difficult part is **how to calculate** `count(x)`, because now both arrays can contain **negative, zero, and positive numbers**.

Let's build it from scratch.

## 1. The main idea is still the same

We want the  $K$ -th smallest product.

We cannot generate all products because:

$$5 \times 10^4 \times 5 \times 10^4 = 2.5 \times 10^9$$

products.

So:

Generate all products ❌

Instead:

Guess  $X$   
↓  
How many products  $\leq X$ ?  
↓  
if count  $\geq k$ :  
     $X$  could be answer  
else:  
     $X$  is too small  
↓  
Binary search

So yes, the **same binary-search-on-answer pattern** applies.

## 2. What changes?

In the previous problem:

$$A[i] + B[j] \leq X$$

we could simply rearrange:

$$B[j] \leq X - A[i]$$

That was easy because subtraction does not change the inequality direction.

But now:

$$A[i] \times B[j] \leq X$$

and the problem is:

**What if  $A[i]$  is positive?**

Then:

$$B[j] \leq \frac{X}{A[i]}$$

**What if  $A[i]$  is negative?**

Then dividing by  $A[i]$  **reverses the inequality**:

$$B[j] \geq \frac{X}{A[i]}$$

**What if  $A[i] = 0$ ?**

Then:

$A[i] \times B[j] = 0$

So whether it contributes depends on whether:

$0 \leq X$

This is the main complication.

### 3. This is why negative numbers matter

Let's take:

```
nums1 = [ -4, -2, 0, 3 ]
nums2 = [ 2, 4 ]
```

Suppose:

```
X = -5
```

We want:

$nums1[i] \times nums2[j] \leq -5$

Consider each element of `nums1`.

`a` = -4

We need:

$-4 \times b \leq -5$

Since `-4` is negative, dividing reverses the inequality:

$b \geq \frac{-5}{-4}$

approximately:

$b \geq 1.25$

`nums2` = [2,4]

Both satisfy it:

```
-4 * 2 = -8
-4 * 4 = -16
```

So count = 2.

`a` = -2

We need:

$-2b \leq -5$

Therefore:

$b \geq 2.5$

In [2,4], only 4 works.

```
-2 * 4 = -8
```

Count = 1.

`a` = 0

Every product is:

```
0
```

But:

```
0 <= -5
```

is false.

So count = 0.

`a` = 3

We need:

$3b \leq -5$

Therefore:

$$b \leq -\frac{5}{3}$$

But `nums2` contains only positive numbers.

So `count = 0`.

Total:

```
count(-5) = 2 + 1 + 0 + 0 = 3
```

## 4. So we need three cases

For each `a = nums1[i]`:

```
if a > 0
    count B values such that
    B <= X/a

if a < 0
    count B values such that
    B >= X/a

if a == 0
    product = 0
```

Because `nums2` is sorted, we can use binary search.

This is the key.

## 5. Case 1: `a > 0`

We have:

$$a \times b \leq X$$

Since `a > 0`:

$$b \leq X/a$$

So we need the number of elements in `nums2` satisfying:

$$b \leq X/a$$

That's:

```
upper_bound(nums2.begin(), nums2.end(), X / a)
```

The position returned tells us how many elements are `<= X/a`.

For example:

```
nums2 = [-5, -2, 0, 3, 7]
a = 3
X = 8
```

We need:

$$b \leq 8/3$$

So:

$$b \leq 2$$

Values:

```
[-5, -2, 0]
```

There are 3.

## 6. Case 2: `a < 0`

Now:

$$a \times b \leq X$$

Since `a` is negative, dividing reverses the inequality:

$$b \geq X/a$$

So we need the number of elements:

```
b >= X/a
```

We can use:

```
lower_bound(nums2.begin(), nums2.end(), X / a)
```

lower\_bound gives the first element  $\geq$  target.

Therefore:

```
count += nums2.end() - lower_bound(...);
```

For example:

```
nums2 = [-5, -2, 0, 3, 7]
a = -2
X = 8
```

We need:

$$b \geq 8 / -2 \Rightarrow b \geq -4$$

Elements satisfying:

```
[-2, 0, 3, 7]
```

There are 4.

---

## 7. But there is a BIG issue with integer division

This is where this problem becomes more tricky than your previous problem.

Suppose:

```
X = -5
a = 2
```

Mathematically:

$$X/a = -2.5$$

We need:

$$b \leq -2.5$$

But C++ integer division:

```
-5 / 2
```

gives:

```
-2
```

That's **not** what we want.

Because:

```
b <= -2
```

is different from:

```
b <= -2.5
```

For example:

```
b = -2
```

Then:

$$2 \times (-2) = -4$$

and:

```
-4 <= -5
```

is false.

So we must be careful.

---

## 8. Easiest solution: don't divide

A very clean approach is to use multiplication with binary search.

Instead of trying to calculate:

$$X/a$$

we can binary search over `nums2` and directly check:

```
nums1[i] * nums2[mid] <= X
```

But we have to exploit the sorted order correctly.

There is an even cleaner approach: split `nums1` into:

```
negative
zero
positive
```

and handle each using `lower_bound` / `upper_bound` with carefully computed division.

For interview/competitive programming, another excellent approach is to use `__int128` and binary search indices.

---

## 9. A very robust `count()` approach

For each `a`, binary search `nums2`.

If `a > 0`

As `b` increases:

```
a*b
```

also increases.

So products look like:

```
-20 -10 0 10 20 30
```

We need:

```
product <= X
```

Therefore all valid values are on the **left**.

Find the first invalid position.

---

If `a < 0`

As `b` increases:

```
a*b
```

decreases.

For example:

```
a = -2

b:      -5  -2   0   3   7
product: 10   4   0  -6 -14
```

Now the valid products `<= x` are on the **right**.

So we can binary search for the first valid position.

This avoids all tricky division.

---

## 10. Let's write the counting function

```
long long countLessEqual(
    const vector<long long>& A,
    const vector<long long>& B,
    long long X
)
{
    long long count = 0;
```

```

for (long long a : A) {

    if (a > 0) {

        // Find first b such that a*b > X
        int lo = 0;
        int hi = B.size();

        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;

            if ((__int128)a * B[mid] <= X)
                lo = mid + 1;
            else
                hi = mid;
        }

        count += lo;
    }

    else if (a < 0) {

        // Find first b such that a*b <= X
        int lo = 0;
        int hi = B.size();

        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;

            if ((__int128)a * B[mid] <= X)
                hi = mid;
            else
                lo = mid + 1;
        }

        count += B.size() - lo;
    }

    else {

        // a == 0
        if (X >= 0)
            count += B.size();
    }
}

return count;
}

```

Notice the beauty of this approach:

We don't have to worry about integer division at all.

## 11. Why does this binary search work?

Let's look at:

```
a > 0
```

Suppose:

```

a = 2
B = [-5,-2,0,3,7]
X = 5

```

Products:

```
-10 -4 0 6 14
```

We need:

```
product <= 5
```

So:

```
YES YES YES NO NO
```

This is monotonic.

Therefore binary search works.

---

For:

```
a < 0
```

Suppose:

```
a = -2
B = [-5, -2, 0, 3, 7]
```

Products:

```
10 4 0 -6 -14
```

For:

```
X = 5
```

we get:

```
NO YES YES YES YES
```

Again monotonic.

So binary search works.

---

## 12. Now binary search the answer

The smallest possible product is:

$$-10^5 \times 10^5 = -10^{10}$$

The largest possible product is:

$$10^5 \times 10^5 = 10^{10}$$

So:

```
long long low = -1e10;
long long high = 1e10;
```

Then:

```
while (low < high) {
    long long mid = low + (high - low) / 2;

    if (countLessEqual(A, B, mid) >= k)
        high = mid;
    else
        low = mid + 1;
}
```

Same logic as your previous problem!

---

## 13. Complete code

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

// Count how many products A[i] * B[j] are <= X
ll countLessEqual(
    const vector<ll>& A,
    const vector<ll>& B,
    ll X
) {
    ll count = 0;

    for (ll a : A) {
```

```

// Case 1: a > 0
if (a > 0) {

    // Products increase as B increases.
    // Find first position where a * B[pos] > X.

    int lo = 0;
    int hi = B.size();

    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;

        if ((__int128)a * B[mid] <= X) {
            lo = mid + 1;
        } else {
            hi = mid;
        }
    }

    // [0, lo-1] are valid
    count += lo;
}

// Case 2: a < 0
else if (a < 0) {

    // Products decrease as B increases.
    // Find first position where a * B[pos] <= X.

    int lo = 0;
    int hi = B.size();

    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;

        if ((__int128)a * B[mid] <= X) {
            hi = mid;
        } else {
            lo = mid + 1;
        }
    }

    // [lo, B.size()-1] are valid
    count += B.size() - lo;
}

// Case 3: a == 0
else {

    // 0 * B[j] = 0
    if (X >= 0) {
        count += B.size();
    }
}
}

return count;
}

int main() {

    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, m;
    ll k;

    cin >> n >> m >> k;

    vector<ll> nums1(n);
    vector<ll> nums2(m);

    for (ll &x : nums1)
        cin >> x;

```

```

for (ll &x : nums2)
    cin >> x;

/*
    We iterate over the smaller array.

    This reduces the number of binary searches.
*/
if (nums1.size() > nums2.size()) {
    swap(nums1, nums2);
}

sort(nums1.begin(), nums1.end());
sort(nums2.begin(), nums2.end());

ll low = -1000000000LL;
ll high = 1000000000LL;

while (low < high) {

    ll mid = low + (high - low) / 2;

    ll cnt = countLessEqual(nums1, nums2, mid);

    if (cnt >= k) {
        // There are at least k products <= mid.
        // So answer can be mid or smaller.
        high = mid;
    }
    else {
        // Not enough products <= mid.
        // Answer must be bigger.
        low = mid + 1;
    }
}

cout << low << '\n';

return 0;
}

```

## 14. Let's compare this with your previous problem

This is actually a very good way to understand the progression.

### Previous problem

You had:

$$A[i] + B[j] \leq X$$

Rearrange:

$$B[j] \leq X - A[i]$$

Then:

`upper_bound()`

### Current problem

You have:

$$A[i] \times B[j] \leq X$$

Now there are three cases:

```

A[i] > 0
↓
B[j] <= X / A[i]

A[i] < 0
↓
B[j] >= X / A[i]

A[i] == 0
↓
product = 0

```

And because of the complications of negative integer division, we can instead binary-search `B` using the actual product comparison.

## 15. The most important conceptual difference

Your previous problem had a very simple `count()`:

```

for each A[i]
    find how many B[j] <= X - A[i]

```

This problem has:

```

for each A[i]

    if positive:
        products increase
        count valid prefix

    if negative:
        products decrease
        count valid suffix

    if zero:
        all products are zero

```

So the **outer binary search is the same**.

The **counting function is different**.

That's exactly what you should learn from these problems.

## 16. One more important point: `__int128`

Why did I write:

```
(__int128)a * B[mid]
```

instead of:

```
a * B[mid]
```

Even though the answer is around  $10^{10}$ , using `__int128` makes the multiplication comparison completely safe.

For example:

```

a = 100000
b = 100000

```

Product:

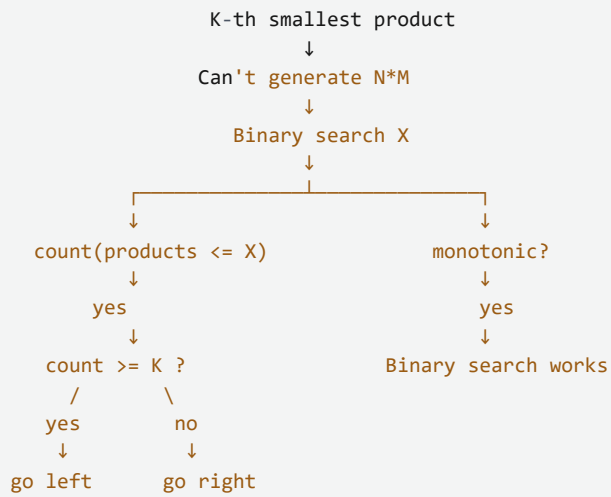
$10^{10}$

`long long` can easily hold that, but using `__int128` makes the multiplication safe and explicit, especially when constraints change or intermediate calculations become larger.

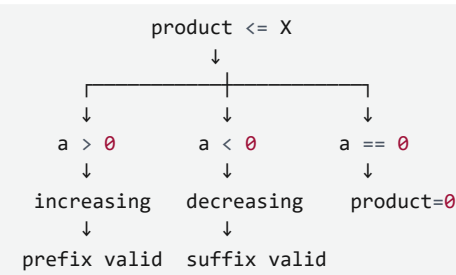
## 17. The pattern you should remember

This problem is a **more advanced version** of the one you just solved.

The pattern is:



And the new challenge is:



So yes: same fundamental logic, but a much more complicated check/count function because multiplication + negative numbers changes the ordering.